



Урок 6. Подзапросы

1. Что такое подзапрос?
2. Подзапросы в FROM
3. Подзапросы в JOIN
4. Подзапросы в SELECT
5. Подзапросы в WHERE
6. Подзапросы в HAVING
7. Подзапросы в ORDER BY
8. Common Table Expression (CTE)
9. Вопросы на собеседованиях
10. Резюме

Урок 6. Подзапросы

Что такое подзапрос?

Подзапрос — это запрос внутри другого запроса. Например, мы можем посчитать агрегат по одной таблице, а затем использовать полученные данные в другом запросе к другой таблице. Подзапросы могут использоваться в различных частях основного запроса, таких как `SELECT` , `FROM` , `JOIN` , `WHERE` , `HAVING` и `ORDER BY` .

Типы подзапросов

1. **Некоррелированные подзапросы** — это подзапросы, которые могут быть выполнены независимо от внешнего запроса. Такой подзапрос выполняется один раз, и его результат используется во внешнем запросе.

Предположим, у нас есть таблица `orders` (заказы) и таблица `customers` (клиенты). Таблица `orders` содержит информацию о заказах, включая ID заказа, ID клиента и сумму заказа. Таблица `customers` содержит информацию о клиентах, включая ID клиента и имя клиента.

```
1 SELECT name
2   FROM customers
3  WHERE customer_id IN (
4      SELECT customer_id
5      FROM orders
6      WHERE order_amount > 1000
7 );
```

Приведенный выше запрос выбирает имена клиентов, которые сделали заказы на сумму более 1000. Подзапрос `SELECT customer_id FROM orders WHERE order_amount > 1000` выполняется один раз и возвращает список ID клиентов, которые затем используются во внешнем запросе.

2. **Коррелированные подзапросы** — это подзапросы, которые обращаются к внешней таблице, то есть они зависят от внешнего запроса. Для каждой строки во внешнем запросе подзапрос возвращает результат, зависящий от значения этой строки. Таким образом, коррелированный подзапрос выполняется многократно, по одному разу для каждой строки внешнего запроса, что влияет на производительность.

Предположим, у нас есть две таблицы: `employees` (сотрудники) и `departments` (отделы). Таблица `employees` содержит информацию о сотрудниках, включая их ID, имя, зарплату и ID отдела. Таблица `departments` содержит информацию об отделах, включая ID отдела и имя отдела.

```
1 SELECT e1.name, e1.salary
2   FROM employees e1
3  WHERE e1.salary > (
4      SELECT AVG(e2.salary)
5      FROM employees e2
6      WHERE e2.department_id = e1.department_id
```

```
7 );
```

Приведенный выше запрос выбирает сотрудников, чья зарплата выше средней зарплаты в их отделе. Подзапрос внутри `WHERE` выполняется для каждой строки из внешнего запроса `employees e1`.

Также в отдельную группу выделяют **скалярные запросы**. Это запросы, которые возвращают единственное значение. Например, `SELECT AVG(column) FROM table;`.

Подзапросы в FROM

```
1 SELECT *
2 FROM (SELECT * FROM table) AS t1;
```

Это некоррелированный подзапрос, в котором внешний `SELECT` получает данные из внутреннего подзапроса.



Некоторые СУБД требуют, чтобы у подзапросов, которые используются во `FROM`, были алиасы. Использование алиасов — полезная привычка!

Пример подзапроса в `FROM`:

```
1 SELECT substr(episode_id, 1, 3) AS season, COUNT(DISTINCT name) AS char_cnt
2 FROM
3     (SELECT e.episode_id, c.name
4      FROM episodes AS e
5      JOIN char_ep AS ce
6          ON e.id = ce.episode_id
7      JOIN characters AS c
8          ON ce.character_id = c.id
9      ) AS epis_char
10 GROUP BY substr(episode_id, 1, 3)
11 ORDER BY 1
```

Table

[+ Add Visualization](#)

season	char_cnt
S01	188
S02	167
S03	195
S04	177
S05	162

Подзапросы в JOIN

```
1 SELECT *
2 FROM table1 AS t1
3 JOIN (SELECT * FROM table2) AS t2
4 ON t1.id = t2.id;
```

Использование подзапроса в секции **JOIN** позволяет объединять таблицу с результатами подзапроса.

Усложним предыдущий запрос и добавим к нему подзапрос в **JOIN :**

```
1 SELECT epis_char.season,
2     COUNT(DISTINCT ep.episode_id) AS ep_cnt,
3     COUNT(DISTINCT epis_char.name) AS char_cnt
4 FROM
5     (SELECT substr(e.episode_id, 1, 3) AS season, c.name
6     FROM episodes AS e
7     JOIN char_ep AS ce
8         ON e.id = ce.episode_id
9     JOIN characters AS c
10        ON ce.character_id = c.id
11     ) AS epis_char
12 JOIN episodes AS ep ON substr(ep.episode_id, 1, 3) = epis_char.season
13 GROUP BY epis_char.season
```

Table

+ Add Visualization

season	ep_cnt	char_cnt
S01	11	188
S02	10	167
S03	10	195
S04	10	177
S05	10	162

Подзапросы в SELECT

Некоррелированный подзапрос:

```

1 SELECT id, name, (SELECT avg(price) FROM table2) AS new_column
2 FROM table;
```

Выше мы используем подзапрос, чтобы добавить еще одно поле в основной запрос. Подзапрос должен быть скалярным, иначе СУБД не поймет, какое из значений, возвращенных подзапросом, необходимо использовать в конкретной строке, и вернет ошибку.



Некоррелированный подзапрос в `SELECT` должен быть скалярным.

Коррелированный подзапрос:

```

1 SELECT name,
2       (SELECT COUNT(1)
3        FROM orders
4        WHERE orders.employee_id = employees.employee_id) AS order_cnt
5 FROM employees;
```

В запросе выше подзапрос использует данные из внешнего запроса для фильтрации, которая объединяет внутреннюю и внешнюю таблицы.

Пример подзапроса в **SELECT** :

▼

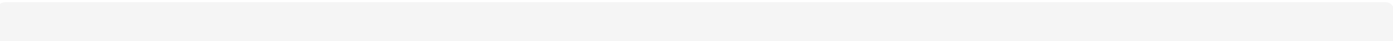
```
1 SELECT epis_char.season,
2     (SELECT COUNT(ep.episode_id)
3       FROM episodes AS ep
4       WHERE substr(ep.episode_id, 1, 3) = epis_char.season) AS ep_cnt,
5     (SELECT COUNT(1) FROM episodes) AS all_ep_cnt,
6     COUNT(DISTINCT epis_char.name) AS char_cnt
7 FROM
8     (SELECT substr(e.episode_id, 1, 3) AS season, c.name
9       FROM episodes AS e
10      JOIN char_ep AS ce
11        ON e.id = ce.episode_id
12      JOIN characters AS c
13        ON ce.character_id = c.id
14     ) AS epis_char
15 GROUP BY epis_char.season
16
```

Table + Add Visualization

season	ep_cnt	all_ep_cnt	char_cnt
S01	11	51	188
S02	10	51	167
S03	10	51	195
S04	10	51	177
S05	10	51	162

Подзапросы в WHERE

Некоррелированный подзапрос



```
1 SELECT *
2   FROM table1
3  WHERE price > (SELECT AVG(price) FROM table2);
```

В запросе выше производится фильтрация по значению-константе, возвращаемому скалярным подзапросом (выбираем из таблицы все товары, у которых цена выше средней).

Коррелированный подзапрос

```
1 SELECT *
2   FROM product_price AS pp
3  WHERE
4      pp.price = (
5          SELECT min(ppm.price)
6          FROM product_price AS ppm
7          WHERE ppm.product_id = pp.product_id
8      )
9  ORDER BY pp.product_id, pp.price, pp.store_id;
```

В запросе выше отбираются только записи с минимальной ценой для каждого продукта. Фильтрация осуществляется таким образом, чтобы цена во внешней таблице равнялась минимальной цене во внутренней таблице при равенстве `product_id`.

Многоколоночный подзапрос

```
1 SELECT employee_id, name, department_id
2   FROM employees
3  WHERE (department_id, location) =
4      (SELECT department_id, location
5       FROM departments
6       WHERE department_id = 101);
```

Многоколоночный подзапрос в `WHERE` используется для сравнения двух и более полей с подзапросом. В примере выше мы сравниваем ID департамента и его местоположение с тем, что возвращает нам отфильтрованный подзапрос. Это пример некоррелированного подзапроса.

Инструкции в WHERE для подзапросов с одним полем и несколькими записями:

- **IN** — проверяет, содержится ли значение в результатах подзапроса. Возвращает **TRUE**, если значение поля входит в результат подзапроса, иначе **FALSE**.

```
1 SELECT *
2   FROM table1
3  WHERE id IN (SELECT id FROM table2);
```

Выше пример некоррелированного подзапроса. Во внешнем запросе производится фильтрация, которая оставляет только записи с **id**, входящими в результат подзапроса.

- **EXISTS** — проверяет наличие строк в подзапросе. Возвращает **TRUE**, если подзапрос возвращает хотя бы одну строку, иначе **FALSE**.

```
1 SELECT *
2   FROM table1
3  WHERE EXISTS (SELECT id
4                 FROM table2
5                 WHERE table2.name = table1.name);
```

Из таблицы 1 выбираются данные, для которых в таблице 2 есть **id**, у которых имя в таблице 2 равно имени в таблице 1. Другими словами, мы джойним таблицы и смотрим, есть ли во второй таблице хотя бы один нужный нам ID, и если есть — условие считается выполненным. EXISTS имеет смысл, когда запрос коррелированный.

- **ANY/SOME** — используется для сравнения значения с любым значением из подзапроса. Возвращает **TRUE**, если условие выполняется хотя бы для одного значения в наборе, иначе **FALSE**.

```
1 SELECT employee_id, name, salary
2   FROM employees
3  WHERE salary > ANY (SELECT salary FROM employees WHERE department_id = 10);
```

В запросе выше мы фильтруем записи таким образом, чтобы зарплата была больше, чем хотя бы у одного из сотрудников департамента 10.

ANY и **SOME** – это совершенно одинаковые инструкции, их работа ничем не отличаются. Некоторые СУБД обрабатывают обе инструкции, а некоторые обрабатывают только одну из них — либо только **ANY**, либо только **SOME**.

- **ALL** — используется для сравнения значения со всеми значениями в подзапросе.

Возвращает **TRUE**, если условие выполняется для всех значений в наборе, иначе **FALSE**.

```
1 SELECT employee_id, name, salary
2   FROM employees
3  WHERE salary > ALL (SELECT salary FROM employees WHERE department_id = 10);
```

В запросе выше мы фильтруем записи таким образом, чтобы зарплата была больше, чем у каждого из сотрудников департамента 10.

Пример подзапроса в WHERE :

```
1 SELECT *
2   FROM characters
3  WHERE EXISTS (SELECT 1
4                  FROM char_loc
5                 WHERE location_id = (SELECT id FROM locations WHERE name =
6                                     'Earth (C-137)')
6                  AND char_loc.character_id = characters.id);
```

id	name	status	species	type	gender	origin_id
38	Beth Smith	Alive	Human		Female	1
45	Bill	Alive	Human		Male	1
71	Conroy	Dead	Robot			20
82	Cronenberg Rick		Cronenberg		Male	12
83	Cronenberg Morty		Cronenberg		Male	12
92	Davin	Dead	Human		Male	1

Подзапросы в HAVING

При использовании подзапроса в блоке **HAVING** выбираются только те записи, агрегаты которых есть в подзапросе, или результат сравнения с подзапросом возвращает **TRUE** (подзапрос при этом должен возвращать только одно значение). Пример некоррелированного подзапроса в блоке **HAVING** :

```
1 SELECT id, SUM(price) AS sm
2   FROM table1
3  GROUP BY id
4 HAVING SUM(price) > (SELECT AVG(price) FROM table1);
```

Пример подзапроса в **HAVING** :

```
1 SELECT substr(episode_id, 1, 3) AS season,
2        COUNT(1) AS cnt
3   FROM episodes
4  GROUP BY substr(episode_id, 1, 3)
5 HAVING COUNT(1) < (SELECT COUNT(1) FROM episodes WHERE episode_id like
   'S01%');
```

Table [+ Add Visualization](#)

season	cnt
S03	10
S04	10
S02	10
S05	10

Подзапросы в ORDER BY

Использование коррелированного подзапроса в блоке **ORDER BY** позволяет для каждой строки вычислить значение для сортировки.

```
1 SELECT employee_id, name, salary, department_id
2   FROM employees
3  ORDER BY (SELECT AVG(salary)
```

```
4         FROM employees AS e2
5         WHERE e2.department_id = employees.Department_id);
```

Запрос выше извлекает информацию о сотрудниках и сортирует результаты в зависимости от среднего значения зарплат по отделам, к которым эти сотрудники принадлежат. Подзапрос выполняется для каждой строки основного запроса и вычисляет среднюю зарплату (`AVG(salary)`) для всех сотрудников, работающих в том же отделе, что и текущий сотрудник в строке основного запроса.



Использование некоррелированных подзапросов в блоке `ORDER BY` не имеет практического смысла, так как сортировка по константе не изменит порядок строк, итоговый порядок будет определяться другими факторами, такими как порядок появления строк в исходной таблице, если он определен, или же порядок, в котором строки возвращает сервер базы данных.

Пример подзапроса в `ORDER BY` :

```
1 SELECT *
2   FROM characters
3  ORDER BY (
4      SELECT COUNT(1)
5      FROM char_ep
6      WHERE char_ep.character_id = characters.id
7  ) DESC;
```

Table

+ Add Visualization

id	name	status	species	type	gender	origin_id
1	Rick Sanchez	Alive	Human		Male	1
2	Morty Smith	Alive	Human		Male	
4	Beth Smith	Alive	Human		Female	20
3	Summer Smith	Alive	Human		Female	20
5	Jerry Smith	Alive	Human		Male	20
180	Jessica	Alive	Human	Time God	Female	20

Common Table Expression (CTE)

CTE (Common Table Expression) позволяет вынести подзапрос за пределы основного запроса. Он объявляется с помощью ключевого слова **WITH** и может использоваться несколько раз, но только в рамках одного общего запроса.

```
1 WITH cte_name AS (  
2     SELECT id, name, count FROM table  
3 )  
4 SELECT *  
5 FROM table1  
6 JOIN cte ON table1.id = cte.id;
```

В примере выше после инструкции **WITH** задается название CTE, которое должно быть осмысленным для понимания содержимого запроса. Внутри CTE помещается запрос, результаты которого в дальнейшем могут быть использованы во **FROM**, **JOIN** и других операторах.

Преимущества CTE

- Улучшение читаемости:** Разделение сложного запроса на более понятные части.
- Оптимизация:** Возможность использовать результаты CTE несколько раз без повторного выполнения подзапроса.

Пример оптимизации с CTE

```

1 WITH cte AS (
2     SELECT city, country, date, weather FROM table
3 )
4 SELECT weather
5 FROM cte
6 WHERE country = 'Russia'
7 UNION
8 SELECT weather
9 FROM cte
10 WHERE country = 'Peru';

```

В примере выше подзапрос выполняется СУБД один раз, а используется дважды. Таким образом уменьшается количество ресурсов для выполнения запроса.

СТЕ может быть несколько в одном запросе, в таком случае инструкция **WITH** пишется только один раз, а СТЕ перечисляются через запятую.

```

1 WITH cte AS (
2     SELECT id, name, count FROM table
3 ),
4 cte2 AS (
5     SELECT id, name, SUM(count) FROM cte GROUP BY id, name
6 )
7 SELECT *
8 FROM table1
9 JOIN cte2 ON table1.id = cte2.id;

```

В примере выше второй СТЕ обращается к первому. Это достаточно лаконично и позволяет понять, что происходит в первом СТЕ, что происходит во втором СТЕ (который использует результаты первого) и что происходит в самом **SELECT**, который построен на основе второго СТЕ.

Разница между СТЕ и временными таблицами

- **Временная таблица** создается пользователем и существует до завершения сессии (завершения соединения с СУБД). Пользователь может пользоваться временной таблицей все время, которое он проводит в сессии.
- **СТЕ** существует только в пределах выполнения запроса. Пользователь может использовать его внутри запроса много раз, но как только запрос вернул нам данные – СТЕ удаляется из памяти СУБД.

Пример с использованием CTE:

```
1 WITH epis_char AS
2   (SELECT substr(e.episode_id, 1, 3) AS season, c.name
3     FROM episodes AS e
4     JOIN char_ep AS ce
5       ON e.id = ce.episode_id
6     JOIN characters AS c
7       ON ce.character_id = c.id
8   )
9   SELECT epis_char.season,
10  (SELECT COUNT(ep.episode_id)
11   FROM episodes AS ep
12   WHERE substr(ep.episode_id, 1, 3) = epis_char.season) AS ep_cnt,
13  (SELECT COUNT(1) FROM episodes) AS all_ep_cnt,
14  COUNT(DISTINCT epis_char.name) AS char_cnt
15 FROM epis_char
16 GROUP BY epis_char.season
17 ORDER BY 1;
```

Table

+ Add Visualization

season	ep_cnt	all_ep_cnt	char_cnt
S01	11	51	188
S02	10	51	167
S03	10	51	195
S04	10	51	177
S05	10	51	162

Вопросы на собеседованиях

1. Чем отличаются коррелированные и некоррелированные подзапросы?

- Некоррелированные подзапросы могут выполняться независимо от внешнего запроса
- Коррелированные подзапросы зависят от данных из внешнего запроса

2. Как работают EXISTS, SOME, ALL и ANY?

- EXISTS проверяет наличие хотя бы одной строки в подзапросе
- SOME и ANY сравнивают значение поля с любым значением из подзапроса
- ALL сравнивает значение поля со всеми значениями в подзапросе

3. Что такое скалярный подзапрос?

- Подзапрос, который возвращает единственное значение

4. Что будет, если подзапрос в WHERE вернет NULL?

- Условие не будет выполнено, так как NULL не является ни true, ни false. Таким образом, во внешнем подзапросе не вернется ни одной строки

5. Может ли подзапрос в ORDER BY быть некоррелированным?

- Да, технически может, но он не повлияет на сортировку

6. Зачем используется CTE?

- Для улучшения читаемости и для оптимизации запросов

7. Чем CTE отличается от временной таблицы?

- Временная таблица существует до завершения сессии пользователем, а CTE существует только в пределах выполняемого запроса

8. Может ли CTE ссылаться на другой CTE внутри запроса?

- Да, может, это улучшает читаемость

Резюме

- **Подзапрос** — это запрос внутри другого запроса
- **Типы подзапросов:**
 - **Некоррелированные подзапросы** — подзапросы, которые могут быть выполнены независимо от внешнего запроса
 - **Коррелированные подзапросы** — подзапросы, которые обращаются к внешней таблице, то есть они зависят от внешнего запроса
 - **Скалярные запросы** — запросы, которые возвращают единственное значение
- Подзапросы могут использоваться в блоках `SELECT` , `FROM` , `JOIN` , `WHERE` , `HAVING` и `ORDER BY`

- Некоторые СУБД требуют, чтобы у подзапросов, которые используются во **FROM**, были алиасы
- **Инструкции в WHERE с подзапросами:**
 - **IN** — проверяет, содержится ли значение в результатах подзапроса
 - **EXISTS** — проверяет наличие строк в подзапросе
 - **ANY/SOME** — используется для сравнения значения с любым значением из подзапроса
 - **ALL** — используется для сравнения значения со всеми значениями в подзапросе
- **CTE (Common Table Expression)** позволяет вынести подзапрос за пределы основного запроса. Он объявляется с помощью ключевого слова **WITH** и может использоваться несколько раз, но только в рамках одного общего запроса
- **Преимущества CTE:**
 - **Улучшение читаемости:** Разделение сложного запроса на более понятные части
 - **Оптимизация:** Возможность использовать результаты CTE несколько раз без повторного выполнения подзапроса
- **Разница между CTE и временными таблицами:**
 - **Временная таблица** создается пользователем и существует до завершения сессии (завершения соединения с СУБД)
 - **CTE** существует только в пределах выполнения запроса